

CE3003

Microcontroller programming

Multithreaded C Programming (Pthreads)

by
A/P Nicholas Vun

Content

- **Pthreads**
- **Critical Section**
- **Mutex**
- **Semaphore**

Multitasking

With increased complexity in many embedded applications

- operating system (OS) have become an essential element in many modern embedded systems

Multitasking is probably the main advantage for using an OS

- as it means multiple tasks can run 'concurrently'

In a hosted environment (i.e. one running with OS), multiple programs (tasks) can be launched separately

- with their executions managed by the OS using time multiplexing.

However it is often better to have a **single executable** that can **launch multiple tasks**

- which allow the use of shared memory to enable simpler IPC (inter process communication), sharing of data/parameters.

Multithreaded Programming

The most common way to launch multiple tasks within a program is through **multithreaded programming**.

Multithreading separates a process into many execution threads, each of which runs independently.

Some benefits of multithreaded programming:

- better responsiveness for the application
- improved program structure
- less system resources needed
- simpler IPC
- more efficient use of **multi-core** processor

POSIX Threads Standard

POSIX Threads, usually referred to as **Pthreads** is an IEEE standard commonly used for multithreaded programming.

(Aside: POSIX = Portable Operating System Interface, a family of standards specified by the IEEE for maintaining compatibility between operating systems)

Pthreads defines a set of C related types and functions, as well as an API for creating and manipulating threads.

Being a POSIX based standard

- Pthreads is available in many OSs
- programs coded using Pthreads are hence very portable

Multitasking Using Threads

When a program is split into many threads, each thread acts like its own individual program

- except that all the threads work in the same memory space, so all their memory is shared

This makes communication between threads fairly simple.

For example, a media player can have three threads:

- i. a thread for pre-buffering the incoming media, possibly from a harddrive, CD, DVD, or network socket.
- ii. a thread to process user input.
- iii. a thread to play the actual media.

A stall in any single thread won't keep the others from doing their jobs.

Thread Executions

Threads executions are similar to multiple processes (tasks) executions

- time-sliced by the kernel of the OS.

On single core processor systems

- kernel uses time slicing to *simulate* simultaneous execution when running multiple tasks and threads

But on multi-core processor systems, threads can run simultaneously (just like two tasks on multi-core processor)

- i.e. threads will also take advantage of multiple cores.

So if we run a threaded program on a dual-core CPU, it could end up being very efficient.

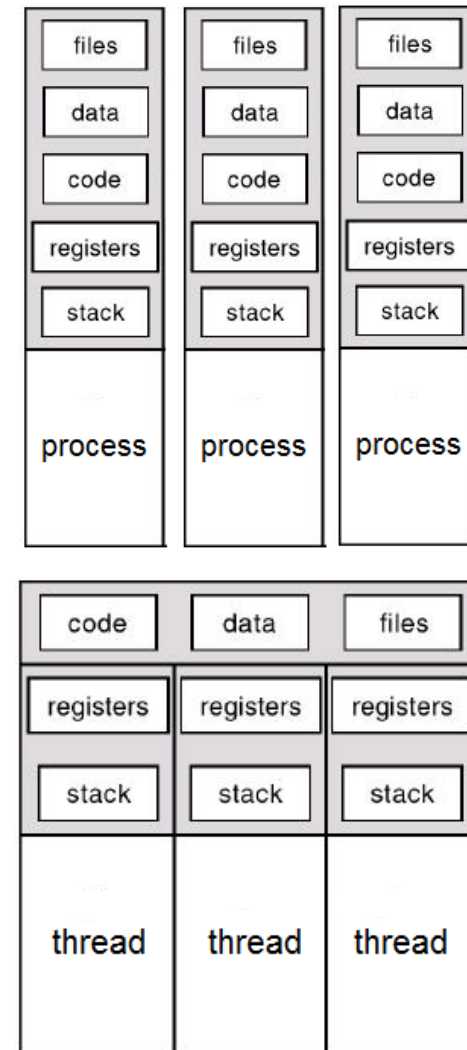
Resources Usage

Threads are also very nimble compared to OS based multitasking process

- the kernel does not need to make a new independent copy of each process memory space, file descriptors, etc. when creating a thread.
- hence a lot less overhead

This also saves a lot of CPU time (10 to 100 times), and enable faster context switching

- creating threads is definitely not going to kill CPU performance!



Multithreading & IPC

As threads share the same memory space

- independent threads can access the same global variables.

POSIX threads hence don't need time consuming and complicated IPC procedures to communicate between threads.

With a little synchronization

- all threads can read and modify a program's data structures

(In this topic, we will learn writing multithreaded C programming using the Pthreads)

Pthreads C Program

A multithreaded C program needs to have the following in its program:

- include the Pthread's header file

#include <pthread.h>

- define the thread using

pthread_t

- create the thread using the function

pthread_create()

- wait for the thread to complete using the function

pthread_join()

A Basic Multithreaded C Program

So how does a Pthreads based multithreaded program looks like?

- the following example shows how Pthreads is included for a simple C program.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void *print_msg( void *ptr )
{ char *text;
  text = (char *) ptr;
  printf("%s \n", text);
}
// continue in next slide
```

Basic Multithreaded C Program (cont.)

```
main()  
{ pthread_t thread1, thread2;  
  char *str1 = "I am thread 1";  
  char *str2 = "I am thread 2";  
  int  T1ret, T2ret;  
  /* Create two threads */  
  T1ret = pthread_create(&thread1, NULL,  
                        print_msg, (void*) str1);  
  T2ret = pthread_create(&thread2, NULL,  
                        print_msg, (void*) str2);  
  
  /* The program now wait for both threads to  
     finish */  
  pthread_join( thread1, NULL);  
  pthread_join( thread2, NULL);  
}
```

Aside: Some Notes on Threads

A **thread** does not know it's **parent**, and all **threads** within a process **share address space**.

They also **share**:

- open files (file descriptors)
- signals and signal handlers
- current working directory
- User and group id

But they have **unique**:

- thread ID
- registers, stack pointer
- stack for local variables
- return addresses
- signal mask
- priority

Return value: **pthread**s functions return "0" if OK.

Race Condition

But sharing of global variable by multiple threads must be carefully synchronized to avoid **race condition**.

Consider the following program:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
int  c = 0;      // global variable
void *fnC()
{  int i,j;
   for (i=0;i<10;i++)
       { c++;    // increment the global variable
         for (j=0; j<5000000;j++);
         printf(" %d",c); fflush(stdout);
       }
}
```

Race Condition (cont.)

```
main()  
{ pthread_t tA, tB;  
  int rtA, rtB;  
  /* Create two threads */  
  if ((rtA=pthread_create(&tA, NULL, &fnC, NULL)) )  
    printf("Thread creation failed: %d\n", rtA);  
  
  if ((rtB=pthread_create(&tB, NULL, &fnC, NULL)) )  
    printf("Thread creation failed: %d\n", rtB);  
  
  /* Wait for both threads to finish */  
  pthread_join(tA, NULL);  
  pthread_join(tB, NULL);  
}
```

Can you see the cause of the potential race condition?

Critical Section

The previous example shows a race condition

- where multiple threads update the value of a global (i.e. shared) variable, without synchronization.

The solution: use **Critical section**.

Critical section is the part of a code

- where a shared resource can only be accessed
- but only by one thread at one time in an *atomic* manner

If a critical section is already entered by one thread

- the critical section is considered **locked** by the thread
- any other thread trying to enter the section will be blocked
- until the lock on that critical section is **released** by the exit of the first thread.

Mutex

Mutex is the simplest and most primitive object use to delimit critical sections

- provides **MUTual EXclusion** between threads
- such that threads behave nicely (i.e. synchronize) to one another.

It is used cooperatively between threads to ensure that

- **only one of the** cooperating **threads** is allowed to **access the shared data** or run certain code section **at a time**
- which guarantee that one thread “excludes” all other threads while it executes the critical code section.

Mutex allows access to the shared data by a thread at any one time

- if that thread successfully **‘locks’** the mutex (change its state) before accessing the data.

Mutex (cont.)

Any thread that successfully locks the mutex becomes its owner.

Until it unlocks the mutex

- any other thread that attempts to lock the mutex will be blocked (and put to sleep by OS).

When the thread owner unlocks the mutex

- control is returned to one waiting thread (could be random choice if more than one is waiting)
- with that thread becoming the new owner of the mutex.

Hence there can be only one owner of a mutex at any one time.

Example of Mutex Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int  c = 0;
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;

void *fnC()
{  int i;
   for (i=0;i<10;i++)
       { pthread_mutex_lock( &mutex1 );
         c++;
         for (j=0; j<50000000;j++);
         printf(" %d",c);
         pthread_mutex_unlock( &mutex1 );
       }
}
```

Mutex Control Functions

The Pthreads standard includes a special type for mutex declaration:

`pthread_mutex_t`

A mutex is initialized by calling the function

`pthread_mutex_init()`

- although previous example uses the macro **PTHREAD_MUTEX_INITIALIZER**
- which set its attributes to default values.

A thread that calls `pthread_mutex_lock()` on a mutex is granted access to the mutex if no other thread owns the mutex

- and becomes the owner of the mutex.

Mutex Control Functions (cont.)

If the mutex is already owned by another thread

- the calling thread is blocked (and put to sleep by the OS.)

A thread releases its ownership by calling

`pthread_mutex_unlock()`

- a thread calling `pthread_mutex_unlock()` must be the owner of the mutex.

When the mutex is no longer needed

- it is destroyed by calling (in the main loop)

`pthread_mutex_destroy()`

Semaphore Operation

Semaphore can be thought of as simple counter

- which holds a current count with value ≥ 0
- that indicates the status of a resource

Any thread can **decrement** the count to **lock** the semaphore

- this is called **waiting** on the semaphore.

Attempting to decrement the count past 0 causes the thread that is calling to 'wait'

- until another thread unlock the semaphore.

Any thread can **unlock** the semaphore by **increment** the count

- this is also called **posting** the semaphore

Posting a semaphore may wake up a waiting thread if there is one present.

Semaphore Operation (cont.)

Hence tasks that use semaphores either signals (wake) or waits

- one task will increase the count
- another task decrease the count.

Semaphore is useful for signaling purpose between tasks

- it provides event notification as well as synchronization

In embedded systems

- semaphores can also be used to signal from an interrupt service routine (ISR) to a task.

Semaphore Controls

Semaphore is NOT part of pthread

- hence we need to add the header file:

`<semaphore.h>`

It defines the `sem_t` type used to declare a semaphore variable, and also provide the following main control functions:

`int sem_init(sem_t *sem, int pshared, unsigned int value);`

- use to initialize the semaphore's value
- pshared argument = 0 for semaphores local to a process

`int sem_wait(sem_t * sem)`

- perform the equivalent operation of decrementing the semaphore value

Semaphore Controls (cont.)

int **sem_post**(sem_t * sem)

- perform the equivalent operation of incrementing the semaphore value

int **sem_getvalue**(sem_t *sem, int *valp);

- gets the current value of semaphore

int **sem_destroy**(sem_t * sem)

- use to properly deallocate a semaphore (no thread would wait for the semaphore)

Code Snippet - Binary Semaphore

```
:
#include <pthread.h>
#include <semaphore.h>

int c = 0;
sem_t sem_mutex;      // i.e. a mutex semaphore
void fnC(char k, int p)
{ int i, temp;
  for (i=0;i<p;i++)
  { sem_wait (&sem_mutex);
    temp=c;
    temp=temp+1;
    c=temp;
    sem_post (&sem_mutex);
  }
}
```

Binary Semaphore (cont.)

```
void *tAf() { fnC('A', 1000000); }
void *tBf() { fnC('B', 500000); }

main()
{ int rtA, rtB;
  pthread_t tA, tB;

  sem_init(&sem_mutex, 0, 1); // initial value = 1
  /* Create two threads , A and B */
  pthread_create( &tA, NULL, &tAf, NULL);
  pthread_create( &tB, NULL, &tBf, NULL);

  /* Wait for both threads to finish */
  pthread_join( tA, NULL);
  pthread_join( tB, NULL);
  sem_destroy(&sem_mutex);
}
```

Producer-Consumer Problem

Counting semaphore is useful for resolving the classical Producer-Consumer synchronization requirement

- also known as the **bounded buffer problem**

This is an example of a multitasking synchronization problem on

- how to pass data from one process to another process

Two processes, the producer and the consumer

- share a common, fixed-size (circular) buffer

Producer generates data and put it into the buffer, and resume generation of its next data

- at the same time, the consumer removes the data from the buffer one piece at a time.

Producer-Consumer Problem (cont.)

The problem is to make sure that

- the producer won't try to add data into the buffer if it is already full

and

- the consumer won't try to remove data when buffer is empty

Hence **synchronization** is needed.

Producer/consumer will also need to wait until buffer/data is available

- hence also needs **event notification**

Code Snippet - Producer

```
void Put(char item)
{
    :
    /* write item to shared buffer; If there
       is no empty buffer, wait */
    sem_wait(&shared.buffer);

    shared.buf[shared.in] = item;

    /* increment the number of data
       now available in buffer */
    shared.in = (shared.in+1)%BUFF_SIZE;

    /* generate signal */
    sem_post(&shared.data);
}
```

Code Snippet - Consumer

```
void Get()
{ int item;

    /* Prepare to get data from buffer;
       if there is no data - wait    */
    sem_wait(&shared.data);

    // Have data - get it
    item = shared.buf[shared.out];

    // Increment the buffer now available
    shared.out = (shared.out+1)%BUFF_SIZE;

    // signal
    sem_post(&shared.buffer);
}
```

Code Snippet - Main

```
main()  
{  pthread_t tP, tC;  
    sem_init(&shared.data, 0, 0);  
    sem_init(&shared.buffer, 0, BUFF_SIZE);  
  
    pthread_create( &tP, NULL, &tProducer, NULL));  
    pthread_create( &tC, NULL, &tConsumer, NULL));  
  
    pthread_join(tP, NULL);  
    pthread_join(tC, NULL);  
  
    sem_destroy(&shared.data);  
    sem_destroy(&shared.buffer);  
}
```


Multiple Producers/Consumers

With **multiple producers** sharing the same buffer for their items, or **multiple consumers** sharing the same memory space

- using **counting semaphores** can **cause a serious race condition** that could result in two or more processes reading or writing into the same slot at the same time.

Example: Two producers decrement the **empty** semaphore

- one of the producers determines the next empty slot in the buffer
- second producer determines the next empty slot and gets the same result as the first producer
- and both producers write into the same slot

Solution: need a way to make sure that **only one thread is executing the writing at a time**

Code Snippet – Mutex with Semaphore

```
void Put(char item)
{ int item;

  sem_wait(&shared.buffer);

  /* Have empty buffer – check whether buffer
    is ready, if not wait */
  sem_wait(&shared.mutex);

  shared.buf[shared.in] = item;
  shared.in = (shared.in+1)%BUFF_SIZE;

  sem_post(&shared.mutex);

  sem_post(&shared.data);
}
```

Comparison

Both mutex and semaphores can be used to provide mutual exclusion for critical section

- but semaphore is more versatile as it can also be used for event notification and thread synchronization

Mutex is robust in providing mutual exclusion and hence is good for critical application

- but semaphore is faster and uses fewer system resources

There is no concept of ownership for semaphore

- semaphore is not considered to be owned by a thread
 - one thread can execute **`sem_wait()`** on a semaphore and another thread can execute **`sem_post()`**.

Compared it with mutex, which is locked and unlocked by the same thread and hence is owned by the thread.

Summary

Multithreaded programming enables efficient use of system resources and allow simple IPC.

But need to be aware of data access synchronization issue that may cause race condition.

Critical section is section of code that only allow one thread to enter at any one time.

Mutex is used to delimit the critical section.

Semaphore can also be used, but is more flexible than mutex, and also allows event notification function.